



ENCRYPTED NOTE KEEPER

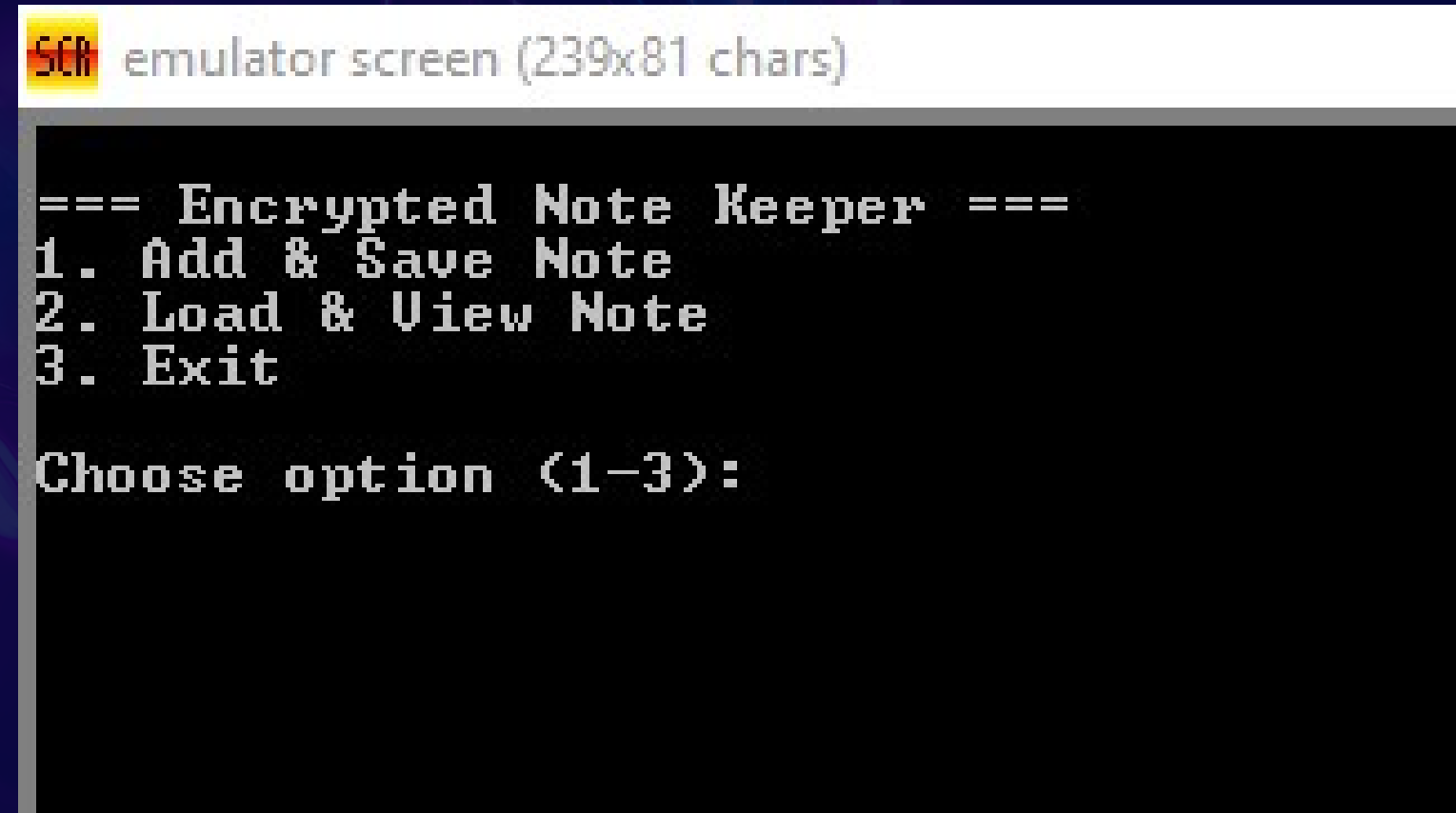
By:
Muhammad Ismail Rana (F2023-551)
Ramsha Naveed (F2023-026)
Saad Mughal (F2023-009)
Sheikh Muhammad Ibrahim (F2023-630)

INTRODUCTION

- Encrypted Note Keeper is an x86 Assembly application for saving XOR-encrypted notes with a 3-digit key. It demonstrates fundamental Assembly concepts like I/O, file handling, and string manipulation, running in 16-bit real mode (DOSBox).
- Encrypted Note Keeper is a basic x86 Assembly application that saves and encrypts notes using XOR with a 3-digit key. It showcases fundamental Assembly operations in a 16-bit DOSBox environment.
- Developed in x86 Assembly, Encrypted Note Keeper is a simple application that saves and encrypts short notes using a 3-digit XOR key. It highlights core Assembly concepts for 16-bit real mode.

Menu Interface

- Menu-Driven Interface: Interactive text menu using DOS int 21h for display and user input (add, view, exit).
- XOR Encryption: Symmetric encryption with a 3-digit key for basic note security.
- Core File Operations: Manages file creation, reading, writing, and closing for notes and keys.
- Modular Design: Code structured into reusable procedures for clarity.

A screenshot of a DOS emulator window titled "emulator screen (239x81 chars)". The window displays a text-based menu for a program called "Encrypted Note Keeper". The menu options are: 1. Add & Save Note, 2. Load & View Note, and 3. Exit. Below the menu, it prompts the user to "Choose option (1-3):".

```
SCM emulator screen (239x81 chars)

=== Encrypted Note Keeper ===
1. Add & Save Note
2. Load & View Note
3. Exit

Choose option (1-3):
```


Adding a Note

- User inputs filename, note, and 3-digit numeric key
- Key converted to number and validated
- XOR encryption performed using a loop (encrypt_note)
- Encrypted data saved to file along with key and length

```
emulator screen (80x25 chars)

=== Encrypted Note Keeper ===
1. Add & Save Note
2. Load & View Note
3. Exit

Choose option (1-3): 1
Enter filename (e.g., notes1.enc): first
Enter your note (max 255 chars): my name is ali
Enter 3-digit key (e.g., 123): 123
Note saved and encrypted successfully!
_
```


Viewing a Note

- User inputs file name and decryption key
- File is read using int 21h (function 3Dh, 3Fh)
- XOR used again to decrypt using same key
- If key matches, note is shown; else, "Wrong key!" is displayed

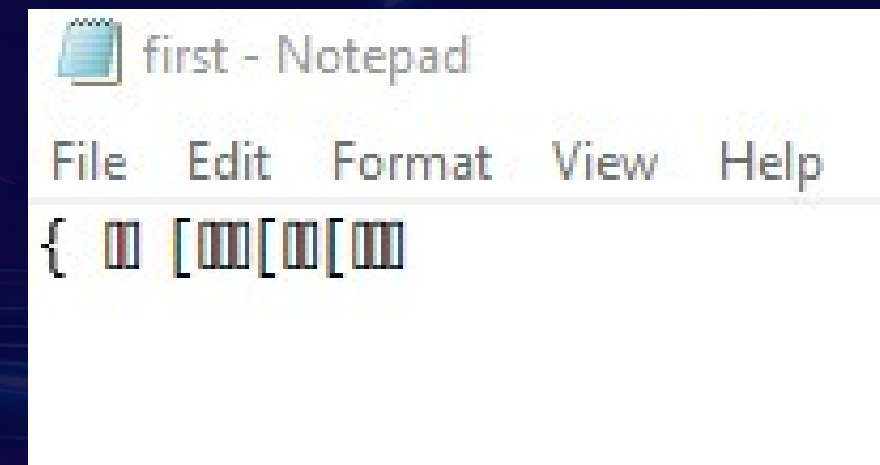
```

emulator screen (80x25 chars)

=== Encrypted Note Keeper ===
1. Add & Save Note
2. Load & View Note
3. Exit

Choose option (1-3): 2
Enter filename to load (e.g., notes1.enc): first
Enter 3-digit key (e.g., 123): 123
Note loaded and decrypted successfully!
my name is ali_

```



- Successfully encrypts and stores notes securely
- Data cannot be read without the correct key

Encryption & Key Logic

- The code iterates through each character of the note.
- In each step, `xor [si], bl` performs a bitwise XOR operation between the current note character (at `[si]`) and the key (`bl`), encrypting the character in place.
- `SI` moves to the next character, and `CX` counts down until all characters are processed.

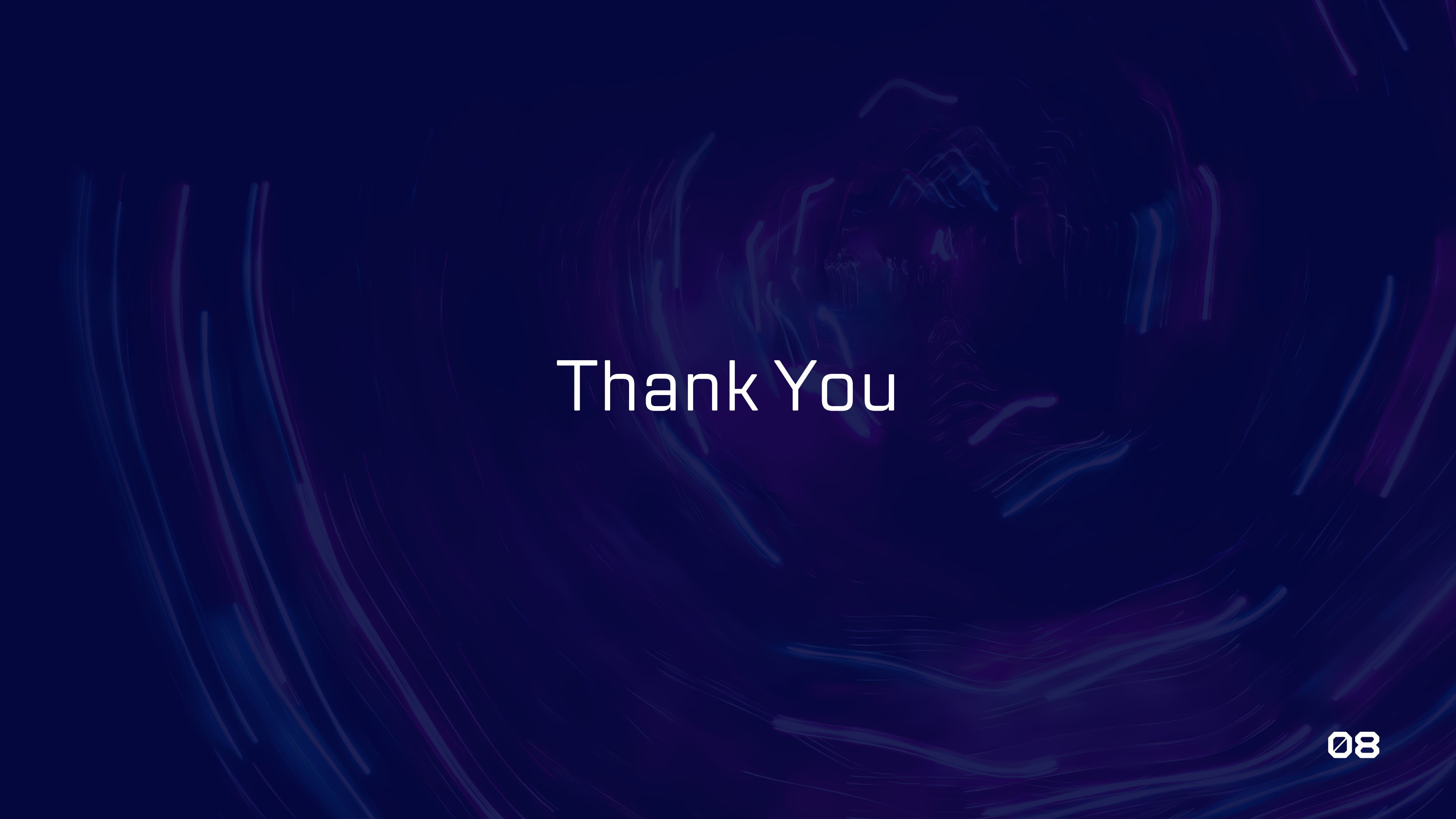
```
encrypt_note proc near
; Encrypts note using XOR with key
    lea si, note_buffer + 2
    mov cl, note_buffer + 1
    mov ch, 0
    mov bx, key

encrypt_loop:
    cmp cx, 0
    je encrypt_done
    xor [si], bl
    inc si
    dec cx
    jmp encrypt_loop

encrypt_done:
    ret
encrypt_note endp
```


Learning Objectives

- Direct System Interaction: How to use BIOS (int 10h) and DOS (int 21h) interrupts for screen control, keyboard input, and file operations.
- Memory and Data Management: Defining data buffers (note_buffer, key_buffer) and variables, and managing memory addresses using registers (DS, DX, SI).
- Core Programming Constructs: Implementing loops, conditional jumps (cmp, je, jmp), and modularizing code with procedures (proc, call, ret).
- Low-Level Data Processing: Performing character-by-character string input/output, converting ASCII to numeric values, and applying bitwise XOR for basic encryption/decryption.



Thank You